# ASSIGNMENT 5

## Encoder-Decoder Based Machine Translation

| Sr. No. | Name of Student | PRN | CO Mapped |
|---|---|---|---|
| 1 | Dhawse Spandan Yashwant | 23070122085 | CO5 |
| 2 | Garvit Tyagi | 23070122094 | CO5 |

| CA3 Project Title | RedactIQ: Explainable PII Detection and Redaction with Confidence-Scored Audit Trails |
|---|---|

## Aim

To implement a sequence-to-sequence encoder-decoder network with Bahdanau attention that translates English sentences into Hindi, and to use it to audit whether personally identifiable information survives machine translation.

The experiment answers a question that a redaction pipeline must settle: if a document is translated, does the personal data in it leak across the language boundary?

**Learning Outcome:**

*After completing this experiment, students will be able to:*

- Implement an encoder-decoder architecture with an attention mechanism in PyTorch and train it with teacher forcing.
- Apply greedy decoding to generate a target sentence and explain the role of the context vector and the attention weights.
- Analyse the behaviour of a neural translation model on personally identifiable information and justify delexicalisation as the correct design for a privacy pipeline.

## Theory

Machine translation maps a sentence in a source language to a sentence in a target language whose length is generally different, so a token-by-token classifier cannot be used. The sequence-to-sequence architecture solves this with two networks. The encoder is a recurrent network that reads the source sentence and produces a hidden state for each token, and the decoder is a second recurrent network that generates the target one word at a time, conditioned on what it has already produced.

In the original formulation the decoder receives only the encoder's final hidden state, a single fixed-size context vector. This is the information bottleneck: one vector of a few hundred numbers has to carry an entire sentence, and translation quality falls sharply as the source grows longer. Bahdanau attention removes the bottleneck. At every decoding step the decoder scores its current state against every encoder state, normalises the scores with a softmax to obtain attention weights, and takes the weighted average of the encoder states as a fresh context vector. The decoder can therefore look back at the specific source words it needs while producing each target word. Padding positions are masked to a large negative value before the softmax so that the model never attends to them.

Training uses teacher forcing: at each step the decoder is fed the true previous target word rather than its own prediction, which keeps training stable and lets the whole sequence be scored in parallel. At inference time the true word is unavailable, so the model is decoded greedily by taking the highest-scoring word at each step and feeding it back. Beam search, which keeps several partial hypotheses

instead of one, generally produces better output at higher cost. The loss is a cross-entropy over the flattened sequence that ignores padding; scoring each time step separately fails, because a time step that is padding for every sentence in the batch contains no valid target and yields an undefined loss.

The interaction between translation and personal data is the point of this experiment. A word-level translation model has a closed vocabulary, so every distinct e-mail address, telephone number and identifier in the training data becomes its own vocabulary entry. The model memorises those strings, and at test time it emits a memorised identifier belonging to a different person rather than copying the one it was given. That is simultaneously a translation error and a privacy failure.

Delexicalisation is the standard remedy and is what production systems use for named entities. Before translation every PII item is replaced by a typed placeholder such as the name, e-mail or phone tag; the model is trained and run entirely on placeholders, so its vocabulary stays small and no identifier ever enters it; after decoding, the original values are substituted back. The identifier bypasses the network completely, which guarantees exact copying and means the model never memorises personal data at all.

## Algorithm

1. Define a set of parallel English and Hindi templates, with Hindi written in romanised form, and generate each sentence pair in two versions: a lexicalised version carrying real identifiers and a delexicalised version in which every PII item is replaced by a typed placeholder.
2. Build a source and a target vocabulary for each version, adding the special tokens for padding, start of sequence, end of sequence and unknown, and compare the two vocabulary sizes.
3. Encode every sentence as a padded index sequence, prefixing the target with the start token and appending the end token.
4. Define the encoder as an embedding layer followed by a GRU that returns one hidden state per source token.
5. Define the attention module, which projects the concatenated decoder and encoder states, scores them, masks the padded positions and returns the softmax-weighted context vector.
6. Define the decoder, which at each step computes the context vector, feeds it together with the embedded previous word to a GRU, and projects the result onto the target vocabulary.
7. Train both models with teacher forcing and the Adam optimiser, computing one cross-entropy loss over the flattened sequence with padding ignored, and clipping the gradient norm to guard against exploding gradients.
8. Decode greedily by taking the highest-scoring word at each step and feeding it back, stopping at the end-of-sequence token.
9. Implement the full pipeline: detect PII with regular expressions, replace it with placeholders, translate, and substitute the original values back into the output.
10. Audit both approaches on held-out sentences by counting how many source PII items appear correctly in the translation, how many are replaced by a different identifier, and how many are lost.

## Program

```python
import re
import random
import torch
import torch.nn as nn

random.seed(42)
torch.manual_seed(42)

# ---------------- Step 1 : Parallel corpus (English -> Hindi) ----------------
# Hindi is romanised so the output stays legible in any terminal font.
# Each template exists in two forms: LEXICALISED, which carries the real
# identifier, and DELEXICALISED, in which every PII item is replaced by a typed
# placeholder. The two are compared in Step 6.
NAMES = ["rohan mehta", "anita desai", "kabir rao", "priya sharma"]

def phone():
    return str(random.randint(6, 9)) + "".join(random.choices("0123456789", k=9))

def pan():
    return "".join(random.choices("ABCDEFGH", k=5)) + \
           "".join(random.choices("0123456789", k=4)) + "A"

TEMPLATES = [
    ("please contact {name} today",            "kripaya aaj {name} se sampark karein"),
    ("the email of {name} is {email}",         "{name} ka email {email} hai"),
    ("the mobile number of {name} is {phone}", "{name} ka mobile number {phone} hai"),
    ("the pan of {name} is {pan}",             "{name} ka pan {pan} hai"),
    ("{name} works at the office",             "{name} karyalay mein kaam karta hai"),
    ("the meeting is scheduled next week",     "baithak agle saptah niyat hai"),
    ("please submit the report today",         "kripaya aaj report jama karein"),
]

def sample(delex):
    """Fill one template. delex=True keeps the placeholders instead of values."""
    en, hi = random.choice(TEMPLATES)
    n = random.choice(NAMES)
    vals = {"name": "<name>" if delex else n,
            "email": "<email>" if delex else n.replace(" ", ".") + "@sitpune.edu.in",
            "phone": "<phone>" if delex else phone(),
            "pan": "<pan>" if delex else pan()}
    return en.format(**vals), hi.format(**vals)

lex = [sample(False) for _ in range(3000)]
delex = [sample(True) for _ in range(3000)]
print("1. PARALLEL CORPUS :", len(lex), "sentence pairs, two versions")
for s, t in delex[:2]:
    print(f"   delexicalised  EN : {s}")
    print(f"                  HI : {t}")
print(f"   lexicalised    EN : {lex[1][0]}")
```

```python
# ---------------- Step 2 : Vocabularies ----------------
PAD, SOS, EOS, UNK = 0, 1, 2, 3

def build_vocab(sents):
    v = {"<pad>": PAD, "<sos>": SOS, "<eos>": EOS, "<unk>": UNK}
    for s in sents:
        for w in s.split():
            v.setdefault(w, len(v))
    return v

SRC_L = TGT_L = 14

def encode(s, v, n, eos=False):
    ids = [v.get(w, UNK) for w in s.split()][:n - 1]
    if eos:
        ids = ids + [EOS]
    return ids + [PAD] * (n - len(ids))

def prepare(corpus):
    sv, tv = build_vocab([s for s, _ in corpus]), build_vocab([t for _, t in corpus])
    X = torch.tensor([encode(s, sv, SRC_L) for s, _ in corpus])
    Y = torch.tensor([[SOS] + encode(t, tv, TGT_L, eos=True)[:TGT_L - 1]
                      for _, t in corpus])
    return sv, tv, X, Y

print("\n2. VOCABULARY SIZE")
for name, c in [("lexicalised", lex), ("delexicalised", delex)]:
    sv, tv, _, _ = prepare(c)
    print(f"   {name:<16} source = {len(sv):>4} words, target = {len(tv):>4} words")

# ---------------- Step 3 : Encoder-decoder with attention ----------------
```

```python
class Encoder(nn.Module):
    """Reads the source sentence and returns one hidden state per token."""
    def __init__(self, n, emb=64, hid=128):
        super().__init__()
        self.emb = nn.Embedding(n, emb, padding_idx=PAD)
        self.rnn = nn.GRU(emb, hid, batch_first=True)

    def forward(self, x):
        return self.rnn(self.emb(x))          # all states, final state

class Attention(nn.Module):
    """Bahdanau attention: score every encoder state against the decoder state."""
    def __init__(self, hid=128):
        super().__init__()
        self.W = nn.Linear(hid * 2, hid)
        self.v = nn.Linear(hid, 1, bias=False)

    def forward(self, dec_h, enc_out, mask):
        d = dec_h.unsqueeze(1).expand(-1, enc_out.size(1), -1)
        e = self.v(torch.tanh(self.W(torch.cat([d, enc_out], -1)))).squeeze(-1)
        e = e.masked_fill(~mask, -1e4)        # never attend to padding
        a = torch.softmax(e, dim=1)
        return torch.bmm(a.unsqueeze(1), enc_out).squeeze(1), a

class Decoder(nn.Module):
    """Generates the target one word at a time from the context vector."""
    def __init__(self, n, emb=64, hid=128):
        super().__init__()
        self.emb = nn.Embedding(n, emb, padding_idx=PAD)
        self.attn = Attention(hid)
        self.rnn = nn.GRU(emb + hid, hid, batch_first=True)
        self.fc = nn.Linear(hid * 2, n)

    def step(self, y, h, enc_out, mask):
        ctx, a = self.attn(h[-1], enc_out, mask)
        o, h = self.rnn(torch.cat([self.emb(y).unsqueeze(1),
                                   ctx.unsqueeze(1)], -1), h)
        return self.fc(torch.cat([o.squeeze(1), ctx], -1)), h, a
```

```python
# ---------------- Step 4 : Training with teacher forcing ----------------
def train(corpus, epochs=10, quiet=True):
    torch.manual_seed(42)
    sv, tv, X, Y = prepare(corpus)
    cut = int(0.9 * len(corpus))
    enc_m, dec_m = Encoder(len(sv)), Decoder(len(tv))
    ps = list(enc_m.parameters()) + list(dec_m.parameters())
    opt = torch.optim.Adam(ps, lr=2e-3)
    # One loss over the whole flattened sequence. Scoring each timestep
    # separately gives NaN whenever a timestep is padding for every sentence.
    lossf = nn.CrossEntropyLoss(ignore_index=PAD)
    for ep in range(1, epochs + 1):
        enc_m.train(); dec_m.train()
        tot = 0.0
        for i in range(0, cut, 64):
            xb, yb = X[i:i+64], Y[i:i+64]
            mask = xb != PAD
            enc_out, h = enc_m(xb)
            steps = []
            for t in range(1, yb.size(1)):
                # teacher forcing: feed the true previous word, not the prediction
                logits, h, _ = dec_m.step(yb[:, t-1], h, enc_out, mask)
                steps.append(logits)
            out = torch.stack(steps, 1)
            opt.zero_grad()
            loss = lossf(out.reshape(-1, out.size(-1)), yb[:, 1:].reshape(-1))
            loss.backward()
            torch.nn.utils.clip_grad_norm_(ps, 1.0)   # guard exploding gradients
            opt.step()
            tot += loss.item() * len(xb)
        if not quiet and ep in (1, 4, 7, 10):
            print(f"   epoch {ep:>2}  training loss = {tot/cut:.4f}")
    return enc_m, dec_m, sv, tv, cut

def translate(model, sentence):
    enc_m, dec_m, sv, tv, _ = model
    inv = {i: w for w, i in tv.items()}
    enc_m.eval(); dec_m.eval()
    x = torch.tensor([encode(sentence, sv, SRC_L)])
    mask = x != PAD
    words = []
    with torch.no_grad():
        enc_out, h = enc_m(x)
        y = torch.tensor([SOS])
        for _ in range(TGT_L):
            logits, h, _ = dec_m.step(y, h, enc_out, mask)
            y = logits.argmax(-1)              # greedy decoding
            if y.item() == EOS:
                break
            words.append(inv.get(y.item(), "<unk>"))
    return " ".join(words)

print(f"\n3. TRAINING THE DELEXICALISED MODEL")
m_delex = train(delex, quiet=False)
print(f"   parameters = "
      f"{sum(p.numel() for p in list(m_delex[0].parameters()) + list(m_delex[1].parameters())):,}")
print(f"\n   training the lexicalised model for comparison ...")
m_lex = train(lex)

print("\n4. SAMPLE TRANSLATIONS (delexicalised model)")
for s, t in delex[m_delex[4]:m_delex[4] + 3]:
    print(f"   source    : {s}")
    print(f"   reference : {t}")
    print(f"   model     : {translate(m_delex, s)}")
```

```python
# ---------------- Step 5 : Delexicalise, translate, relexicalise ----------------
PII = {"email": r"[\w.]+@[\w.]+\.\w+", "phone": r"\b[6-9]\d{9}\b",
       "pan": r"\b[A-H]{5}\d{4}A\b",
       "name": r"\b(?:" + "|".join(NAMES) + r")\b"}

def delexicalise(text):
    vals = {}
    for tag, pat in PII.items():
        m = re.search(pat, text)
        if m:
            vals[tag] = m.group()
            text = text[:m.start()] + f"<{tag}>" + text[m.end():]
    return text, vals

def pipeline(text, model):
    holed, vals = delexicalise(text)
    out = translate(model, holed)
    for tag, val in vals.items():                # restore after translation
        out = out.replace(f"<{tag}>", val)
    return holed, out

demo = "the email of kabir rao is kabir.rao@sitpune.edu.in"
holed, out = pipeline(demo, m_delex)
print("\n5. THE DELEXICALISE - TRANSLATE - RELEXICALISE PIPELINE")
print(f"   original       : {demo}")
print(f"   delexicalised  : {holed}")
print(f"   translated     : {translate(m_delex, holed)}")
print(f"   relexicalised  : {out}")
```

```python
# ---------------- Step 6 : Cross-lingual PII leakage audit ----------------
def find_pii(t):
    return sorted((k, m.group()) for k, p in PII.items() if k != "name"
                  for m in re.finditer(p, t))

print("\n6. CROSS-LINGUAL PII LEAKAGE AUDIT")
print(f"   {'Approach':<38}{'Correct':<10}{'Wrong':<9}{'Dropped'}")
for name, fn in [("naive: translate raw text",
                  lambda s: translate(m_lex, s)),
                 ("delexicalise, translate, restore",
                  lambda s: pipeline(s, m_delex)[1])]:
    ok = bad = gone = 0
    for s, _ in lex[2700:2850]:
        want = find_pii(s)
        if not want:
            continue
        got = find_pii(fn(s))
        for p in want:
            if p in got:      ok += 1
            elif got:         bad += 1
            else:             gone += 1
    print(f"   {name:<38}{ok:<10}{bad:<9}{gone}")

red = re.sub(PII["email"], "EMAIL_REDACTED", demo)
print(f"\n   redacted source : {red}")
print(f"   translated      : {pipeline(red, m_delex)[1]}")
print(f"   PII in output   : {find_pii(pipeline(red, m_delex)[1]) or 'none'}")
```

## Output

```
1. PARALLEL CORPUS : 3000 sentence pairs, two versions
   delexicalised  EN : the email of <name> is <email>
                  HI : <name> ka email <email> hai
   delexicalised  EN : the email of <name> is <email>
                  HI : <name> ka email <email> hai
   lexicalised    EN : please submit the report today
```

```
2. VOCABULARY SIZE
   lexicalised      source =  904 words, target =  906 words
   delexicalised    source =   27 words, target =   29 words

3. TRAINING THE DELEXICALISED MODEL
   epoch  1  training loss = 0.9155
   epoch  4  training loss = 0.0021
   epoch  7  training loss = 0.0009
   epoch 10  training loss = 0.0005
   parameters = 242,205

   training the lexicalised model for comparison ...
```

```
4. SAMPLE TRANSLATIONS (delexicalised model)
   source    : please contact <name> today
   reference : kripaya aaj <name> se sampark karein
   model     : kripaya aaj <name> se sampark karein
   source    : the pan of <name> is <pan>
   reference : <name> ka pan <pan> hai
   model     : <name> ka pan <pan> hai
   source    : please submit the report today
   reference : kripaya aaj report jama karein
   model     : kripaya aaj report jama karein
```

```
5. THE DELEXICALISE - TRANSLATE - RELEXICALISE PIPELINE
   original       : the email of kabir rao is kabir.rao@sitpune.edu.in
   delexicalised  : the email of <name> is <email>
   translated     : <name> ka email <email> hai
   relexicalised  : kabir rao ka email kabir.rao@sitpune.edu.in hai
```

```
6. CROSS-LINGUAL PII LEAKAGE AUDIT
   Approach                          Correct   Wrong    Dropped
   naive: translate raw text         12        36       2
   delexicalise, translate, restore  50        0        0

   redacted source : the email of kabir rao is EMAIL_REDACTED
   translated      : kabir rao ka email <email> hai
   PII in output   : none
```

### Explanation of the Output

The vocabulary comparison shows the scale of the problem immediately. The lexicalised corpus needs 904 source words for only seven sentence templates, because every generated identifier becomes its own token, whereas the delexicalised corpus needs 27. Almost the entire vocabulary of the naive model consists of memorised personal data.

The delexicalised model trains to a loss of 0.0005 and reproduces the held-out references exactly, including the position of each placeholder, which confirms that the encoder-decoder and the attention mechanism have learned the word-order transformation between the two languages.

The pipeline block shows the three stages on one sentence. The e-mail address and the name are replaced by placeholders, the placeholder sentence is translated into Hindi with the placeholders correctly repositioned, and the original values are then substituted back to give a fluent translation with the personal data intact and never exposed to the network.

The audit table is the central result. Translating raw text reproduces only 12 identifiers correctly while emitting 36 that belong to a different person and dropping 2. The naive model is therefore not merely inaccurate; it fabricates personal data, which is a privacy failure in its own right. The delexicalised pipeline reproduces all 50 identifiers correctly with no substitutions and no losses, because the identifier is never passed through the model.

The final block confirms that redaction survives translation. When the e-mail address is replaced before translation, the translated output contains a placeholder and no personal data, so a redacted document remains redacted after it has been machine translated.

## Conclusion

In this experiment a sequence-to-sequence encoder-decoder network with Bahdanau attention was implemented in PyTorch and trained with teacher forcing to translate English into romanised Hindi. The attention mechanism was used to remove the fixed-context-vector bottleneck of the original architecture, padded positions were masked before the softmax, and the model was decoded greedily. Two versions of the same corpus were compared in order to audit the behaviour of the model on personal data. The lexicalised model required a source vocabulary of 904 words for seven templates, because every identifier became a vocabulary entry, and at test time it reproduced only 12 of 50 identifiers correctly while emitting 36 that belonged to other people. Delexicalising the input, translating the placeholders and substituting the original values back reproduced all 50 identifiers exactly and reduced the vocabulary to 27 words. The conclusion for RedactIQ is that personally identifiable information must never be passed through a translation model: it is memorised, it is emitted in the wrong place, and a document that has not been redacted leaks its contents across the language boundary. It was also verified that placeholders inserted by the redaction stage survive translation unchanged, so a redacted document stays redacted downstream.